Threads: High-Level Concurrency

TL;DR
Das Erzeugen von Threads über die Klasse `Thread` oder das Interface `Runnable` und das Hantieren mit **`synchronized`** und `wait()`/`notify()` zählt zu den grundlegenden Dingen beim Multi-Threading mit Java. Auf diesen Konzepten bauen viele weitere Konzepte auf, die ein flexibleres Arbeiten mit Threads in Java ermöglichen.
Dazu zählt unter anderem das Arbeiten mit `Lock`-Objekten und dazugehörigen `Conditions`, was **`synchronized`** und `wait()`/`notify()` entspricht, aber feingranulareres und flexibleres Locking bietet.
Statt Threads immer wieder neu anzulegen (das Anlegen von Objekten bedeutet einen gewissen Aufwand zur Laufzeit), kann man Threads über sogenannte Thread-Pools wiederverwenden und über das Executor-Interface benutzen.
Schließlich bietet sich das Fork/Join-Framework zum rekursiven Zerteilen von Aufgaben und zur parallelen Bearbeitung der Teilaufgaben an.
Die in Swing integrierte Klasse `SwingWorker` ermöglicht es, in Swing Berechnungen in einen parallel ausgeführten Thread auszulagern.

Videos

- VL High-Level Concurrency
- Demo Lock-Objekte
- Demo Executor
- Demo Fork/Join
- Demo SwingWorker

Unterlagen

- PDF-Version

Explizite Lock-Objekte

Sie kennen bereits die Synchronisierung mit dem Schlüsselwort **`synchronized`**.

```
// Synchronisierung der gesamten Methode
public synchronized int incrVal() {
    ...
}
```

```
// Synchronisierung eines Blocks (eines Teils einer Methode)
public int incrVal() {
    ...
    synchronized (someObj) {
        ...
    }
    ...
}
```

Dabei wird implizit ein Lock über ein Objekt (das eigene Objekt im ersten Fall, das Sperrobjekt im zweiten Fall) benutzt.

Seit Java5 kann man alternativ auch explizite Lock-Objekte nutzen:

```java
// Synchronisierung eines Teils einer Methode über ein
// Lock-Objekt (seit Java 5)
// Package `java.util.concurrent.locks`
public int incrVal() {
    Lock waechter = new ReentrantLock();
    ...
    waechter.lock();
    ... // Geschützter Bereich
    waechter.unlock();
    ...
}
```

Locks aus dem Paket `java.util.concurrent.locks` arbeiten analog zum impliziten Locken über **`synchronized`**. Sie haben darüber hinaus aber einige Vorteile:

- Methoden zum Abfragen, ob ein Lock möglich ist: `Lock#tryLock`
- Methoden zum Abfragen der aktuellen Warteschlangengröße: `Lock#getQueueLength`
- Verfeinerung `ReentrantReadWriteLock` mit Methoden `readLock` und `writeLock`
  - Locks nur zum Lesen bzw. nur zum Schreiben
- `Lock#newCondition` liefert ein Condition-Objekt zur Benachrichtigung ala `wait`/`notify`: `await`/`signal` => zusätzliches Timeout beim Warten möglich

Nachteile:

- Bei Exceptions werden implizite Locks durch **`synchronized`** automatisch durch das Verlassen der Methode freigegeben. Explizite Locks müssen **durch den Programmierer** freigegeben werden! => Nutzung des **`finally`**-Block!

Demo: lock.* (https://github.com/Programmiermethoden-CampusMinden/Prog2-Lecture/tree/master/lecture/java-classic/src/lock/)

Thread-Management: Executor-Interface und Thread-Pools

Wiederverwendung von Threads

- Normale Threads sind immer Einmal-Threads: Man kann sie nur **einmal** in ihrem Leben starten (auch wenn das Objekt anschließend noch auf Nachrichten bzw. Methodenaufrufe reagiert)
- Zusätzliches Problem: Threads sind Objekte:
  - Threads brauchen relativ viel Arbeitsspeicher
  - Erzeugen und Entsorgen von Threads kostet Ressourcen
  - Zu viele Threads: Gesamte Anwendung hält an
- Idee: Threads wiederverwenden und Thread-Management auslagern => **Executor-Interface** und **Thread-Pool**

Executor-Interface

```
public interface Executor {
    void execute(Runnable command);
}
```

- Neue Aufgaben als Runnable an einen Executor via `execute` übergeben
- Executor könnte damit sofort neuen Thread starten (oder alten wiederverwenden): `e.execute(r);` => entspricht in der Wirkung `(new Thread(r)).start();`

Thread-Pool hält Menge von "Worker-Threads"

- Statische Methoden von `java.util.concurrent.Executors` erzeugen Thread-Pools mit verschiedenen Eigenschaften:
  - `Executors#newFixedThreadPool` erzeugt ExecutorService mit spezifizierter Anzahl von Worker-Threads
  - `Executors#newCachedThreadPool` erzeugt Pool mit Threads, die nach 60 Sekunden Idle wieder entsorgt werden
- Rückgabe: `ExecutorService` (Thread-Pool)

  ```
  public interface ExecutorService extends Executor { ... }
  ```

- `Executor#execute` übergibt Runnable dem nächsten freien Worker-Thread (oder erzeugt ggf. neuen Worker-Thread bzw. hängt Runnable in Warteschlange, je nach erzeugtem Pool)
- Methoden zum Beenden eines Thread-Pools (Freigabe): `shutdown()`, `isShutdown()`, ...

```
MyThread x = new MyThread();    // Runnable oder Thread

ExecutorService pool = Executors.newCachedThreadPool();

pool.execute(x);    // x.start()
pool.execute(x);    // x.start()
pool.execute(x);    // x.start()

pool.shutdown();    // Feierabend :)
```

Demo: executor.ExecutorDemo (https://github.com/Programmiermethoden-CampusMinden/Prog2-Lecture/blob/master/lecture/java-classic/src/executor/ExecutorDemo.java)

Hintergrund (vereinfacht)

Der Thread-Pool reserviert sich "nackten" Speicher, der der Größe von n Threads entspricht, und "prägt" die Objektstruktur durch einen Cast direkt auf (ohne wirkliche neue Objekte zu erzeugen). Dieses Vorgehen ist in der C-Welt wohlbekannt und schnell (vgl. Thema Speicherverwaltung in der LV "Systemprogrammierung"). In Java wird dies durch eine wohldefinierte Schnittstelle vor dem Nutzer verborgen.

Ausblick

Hier haben wir nur die absoluten Grundlagen angerissen. Wir können auch `Callables` anstatt von `Runnables` übergeben, auf Ergebnisse aus der Zukunft warten (`Futures`), Dinge zeitgesteuert (immer wieder) starten, …

Schauen Sie sich bei Interesse die weiterführende Literatur an, beispielsweise die Oracle-Dokumentation oder auch (Ullenboom 2021) (insbesondere den Abschnitt 16.4 "Der Ausführer (Executor) kommt").

Fork/Join-Framework: Teile und Herrsche

Spezieller Thread-Pool zur rekursiven Bearbeitung parallelisierbarer Tasks

- `java.util.concurrent.ForkJoinPool#invoke` startet Task
- Task muss von `RecursiveTask<V>` erben:

```
public abstract class RecursiveTask<V> extends ForkJoinTask<V> {
    protected abstract V compute();
}
```

Prinzipieller Ablauf:

```
public class RecursiveTask extends ForkJoinTask<V> {
    protected V compute() {
        if (task klein genug) {
            berechne task sequentiell
        } else {
            teile task in zwei subtasks:
                left, right = new RecursiveTask(task)
            rufe compute() auf beiden subtasks auf:
                left.fork();          // starte neuen Thread
                r = right.compute();  // nutze aktuellen Thread
            warte auf ende der beiden subtasks: l = left.join()
            kombiniere die ergebnisse der beiden subtasks: l+r
        }
    }
}
```

Demo: forkjoin.ForkJoin (https://github.com/Programmiermethoden-CampusMinden/Prog2-Lecture/blob/master/lecture/java-classic/src/forkjoin/ForkJoin.java)

Swing und Threads

Lange Berechnungen in Listenern blockieren Swing-GUI

- Problem: Events werden durch **einen** *Event Dispatch Thread* (EDT) **sequentiell** bearbeitet
- Lösung: Berechnungen in neuen Thread auslagern
- **Achtung**: Swing ist **nicht Thread-safe**! Komponenten nicht durch verschiedene Threads manipulieren!

Lösung

=> `javax.swing.SwingWorker` ist eine spezielle Thread-Klasse, eng mit Swing/Event-Modell verzahnt.

- Implementieren:
  - `SwingWorker#doInBackground`: Für die langwierige Berechnung (muss man selbst implementieren)
  - `SwingWorker#done`: Wird vom EDT aufgerufen, wenn `doInBackground` fertig ist

- Aufrufen:
  - `SwingWorker#execute`: Started neuen Thread nach Anlegen einer Instanz und führt dann automatisch `doInBackground` aus
  - `SwingWorker#get`: Return-Wert von `doInBackground` abfragen

Anmerkungen

- `SwingWorker#done` ist optional: *kann* überschrieben werden
  - Beispielweise, wenn nach Beendigung der langwierigen Berechnung GUI-Bestandteile mit dem Ergebnis aktualisiert werden sollen
- `SwingWorker<T, V>` ist eine generische Klasse:
  - `T` Typ für das Ergebnis der Berechnung, d.h. Rückgabetyp für `doInBackground` und `get`
  - `V` Typ für Zwischenergebnisse

Demo: misc.SwingWorkerDemo (https://github.com/Programmiermethoden-CampusMinden/Prog2-Lecture/blob/master/lecture/java-classic/src/misc/SwingWorkerDemo.java)

Letzte Worte :-)

- Viele weitere Konzepte
  - Semaphoren, Monitore, …
  - Leser-Schreiber-Probleme, Verklemmungen, …

  => Verweis auf LV “Betriebssysteme” und “Verteilte Systeme”

- **Achtung**: Viele Klassen sind nicht Thread-safe!

  Es gibt aber meist ein “Gegenstück”, welches Thread-safe ist.

  Beispiel Listen:

  - `java.util.ArrayList` ist **nicht** Thread-safe
  - `java.util.Vector` ist Thread-sicher

  => Siehe Javadoc in den JDK-Klassen!

- Thread-safe bedeutet **Overhead** (Synchronisierung)!

Wrap-Up

Multi-Threading auf höherem Level: Thread-Pools und Fork/Join-Framework

- Feingranulareres und flexibleres Locking mit Lock-Objekten und Conditions
- Wiederverwendung von Threads: Thread-Management mit Executor-Interface und Thread-Pools
- Fork/Join-Framework zum rekursiven Zerteilen von Aufgaben und zur parallelen Bearbeitung der Teilaufgaben
- `SwingWorker` für die parallele Bearbeitung von Aufgaben in Swing

Zum Nachlesen

- Ullenboom (2021, Kap. 16)
- Oracle Corporation (2024)
- Urma, Fusco, und Mycroft (2014, Kap. 7.2)

Lernziele

- k3: Umgang mit High-Level-Abstraktionen: Lock-Objekten und Conditions, Executor-Interface und Thread-Pools, Fork/Join-Framework, SwingWorker

---

Quellen

Oracle Corporation. 2024. „The Java Tutorials“. 2024. https://docs.oracle.com/javase/tutorial/.

Ullenboom, C. 2021. *Java ist auch eine Insel*. 16. Aufl. Rheinwerk-Verlag. https://openbook.rheinwerk-verlag.de/javainsel/index.html.

Urma, R.-G., M. Fusco, und A. Mycroft. 2014. *Java 8 in Action: Lambdas, Streams, and Functional-Style Programming*. Manning Publications.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 2ce9f3f (Tooling: use original folder structure for generated pdf's, 2025-04-17)